ComfortRoute CSE 3311 – Iteration 2

Team Members: Liam Crowley, Fallou Samb, Kassidy Satterfield, Mccoy Robinson

GitHub Repository: https://github.com/Kass-ii/CSE-3311-

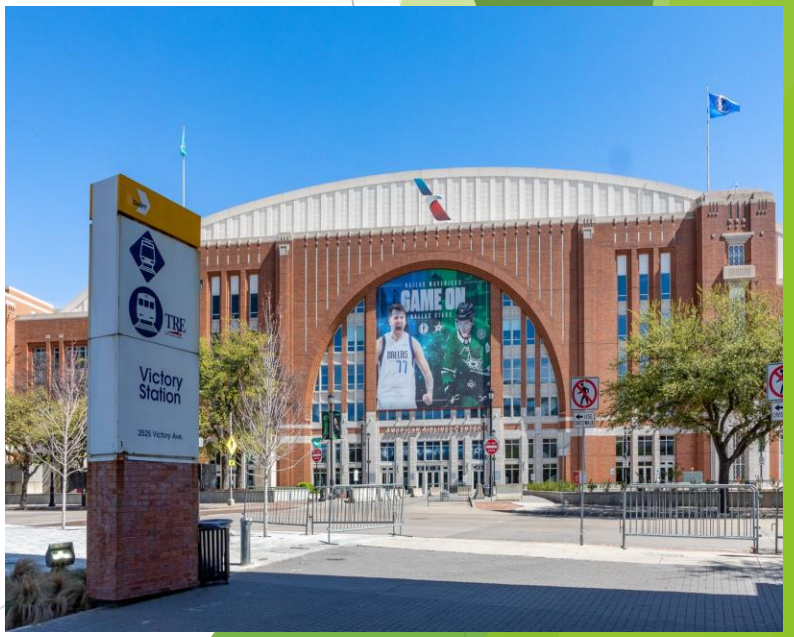
Version: iteration 0.2

User Story

- A user exits a concert at the American Airlines center at 9:20 PM
- The user missed the train back to Fort Worth, the next train is in 90 minutes.
- Rather than wait on the platform for that time they will use ComfortRouter

BOLD TIMES=PM **WEST**BOUND **TRE** WEEKDAY TRAINS (TO FORT WORTH)

Train Number	EBJ Union	Victory	Medical/ Market Ctr.	DT Irving/ Heritage Crossing	West Irving	CentrePort/ DFW Airport	Bell	Trinity Lakes	FW Central Station	FW T&P Station
2959	**9:15**	**9:19**	**9:23**	**9:31**	**9:36**	**9:41**	**9:48**	**9:56**	**10:13**	**10:18**
2961	**10:45**	**10:51**	**10:55**	**11:03**	**11:08**	**11:13**	**11:20**	**11:28**	**11:45**	**11:50**



Iteration 2 Goal

What we set out to build

Out-and-Back Rail Trips

The system allows a user to leave a station, travel to another station using rail transit, and return to the original station before a specified return time.

1. Select an origin station
2. Set departure time
3. Specify latest return time
4. System finds outbound rail trips
5. System finds valid return trips

Example Scenario

Origin	**Centreport station**
Depart	**10:00 AM**
Return by	**10:20 AM**

Centreport → Bell Station → Centreport

Both legs satisfy time constraints

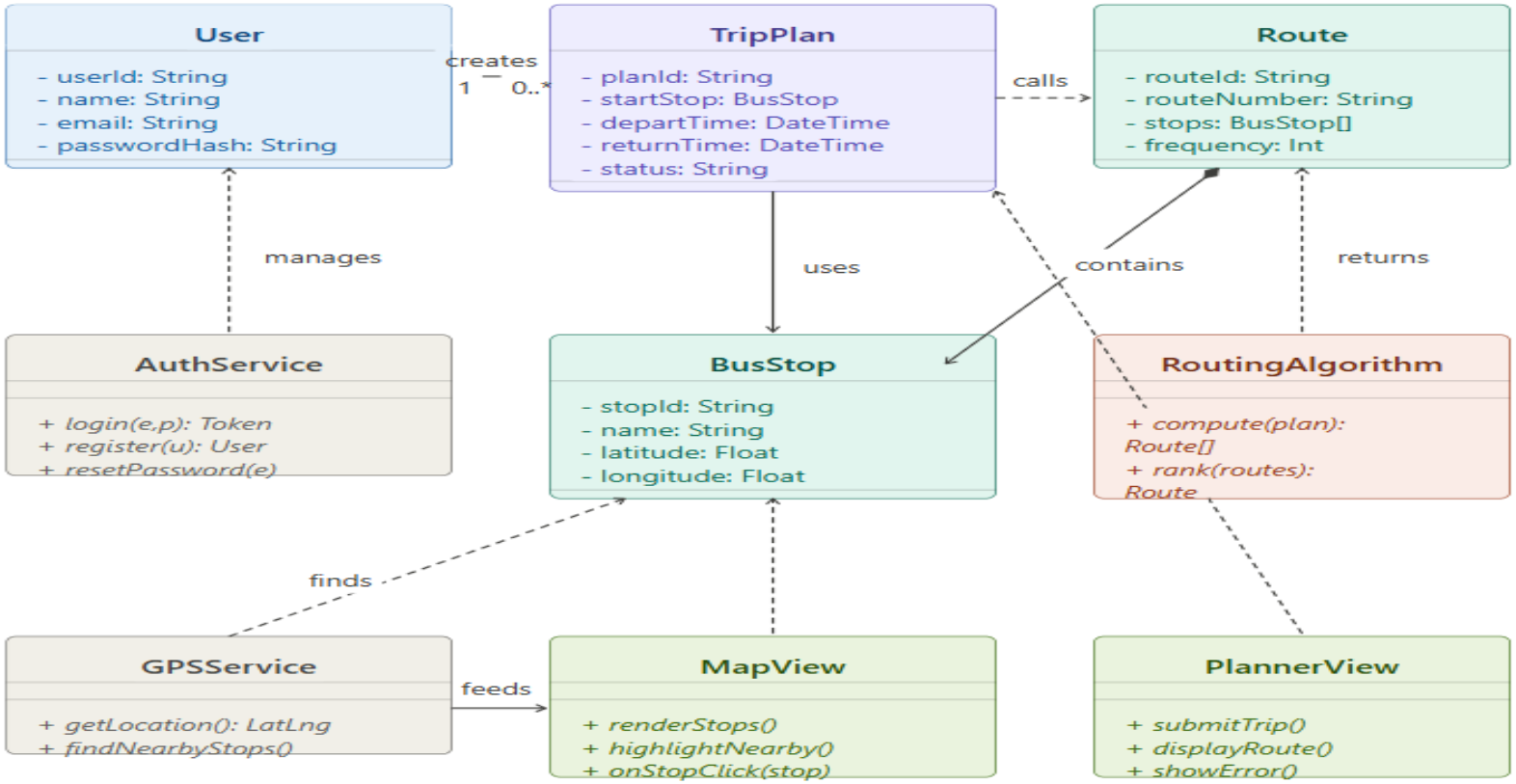
Routing logic/Algorithm

- Load GTFS rail data including stops, trips, and stop_times.
- Identify outbound rail trips from the origin station.
- Determine reachable stations within the time window.
- Search for return trips from those stations back to the origin.
- Select a valid route that satisfies the departure and return constraints.

Added Features: Out-and-back Rail Trips

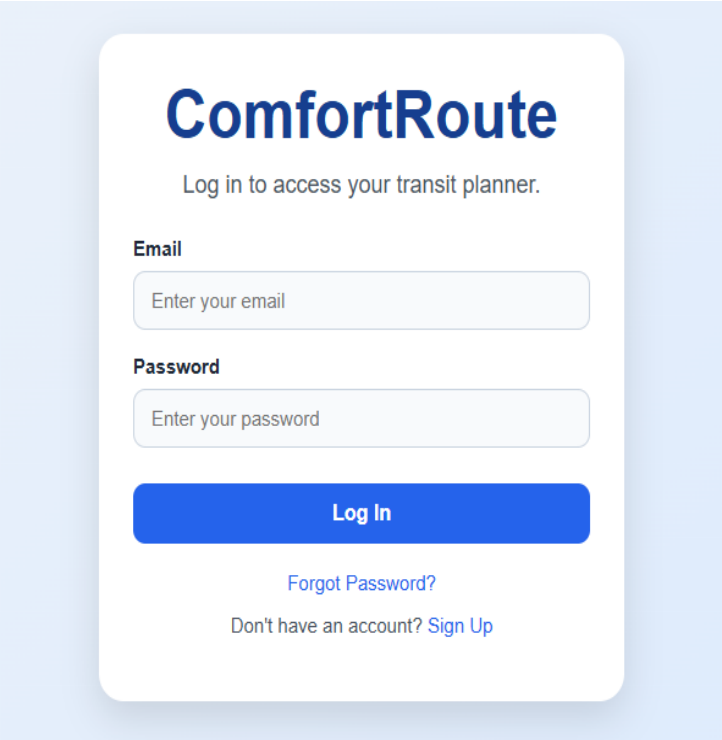
- Workflow:
  - Users can access the app through account creation and logging in.
  - Users will be able to view nearby stations via a transit map.
  - App will take starting station, desired starting and returning time from the user as input to find optimal transit line schedules that satisfy the given constraints.

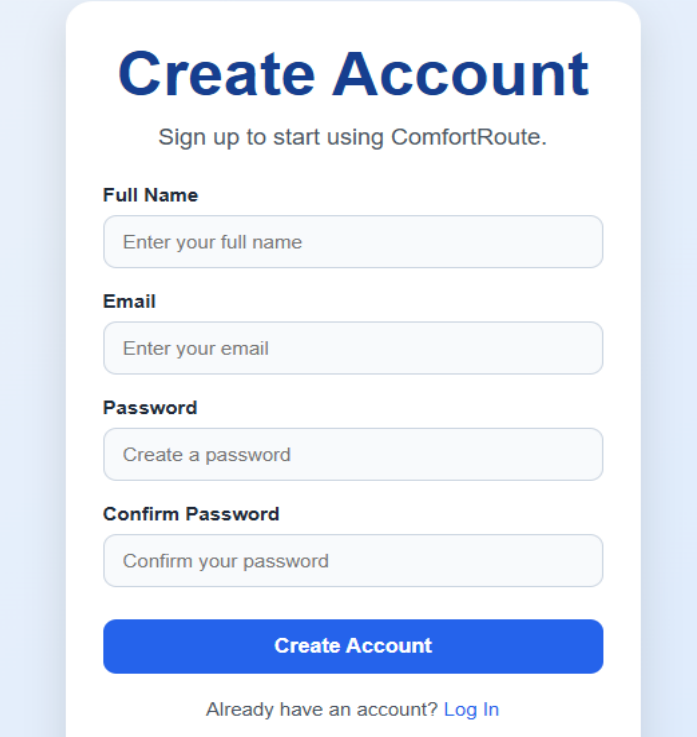
UML Class Diagram



User Authentication

- User is prompted to enter email and password if they have an account.
- Allows new users to create an account before accessing the app.
- Promotes security.

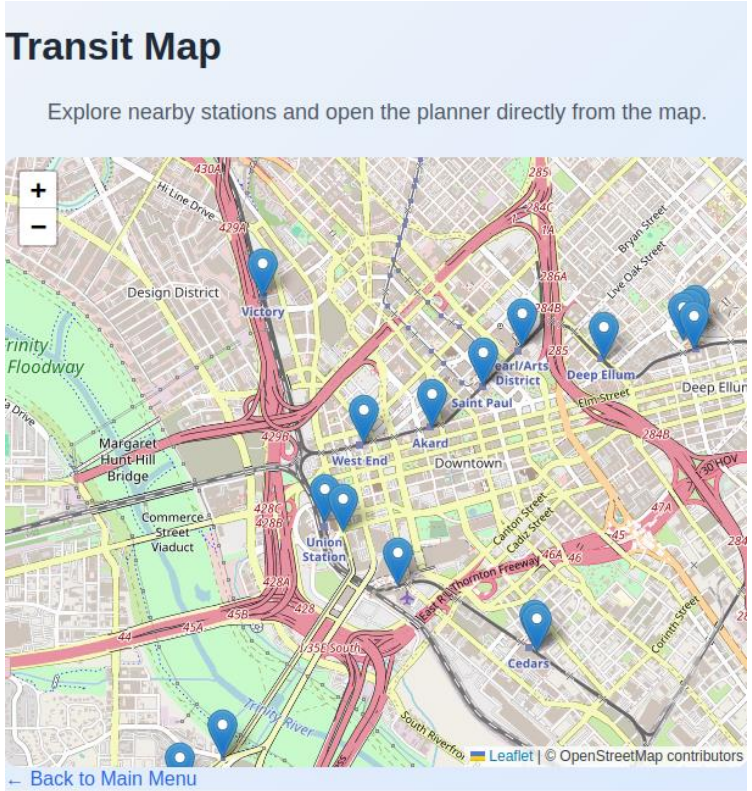


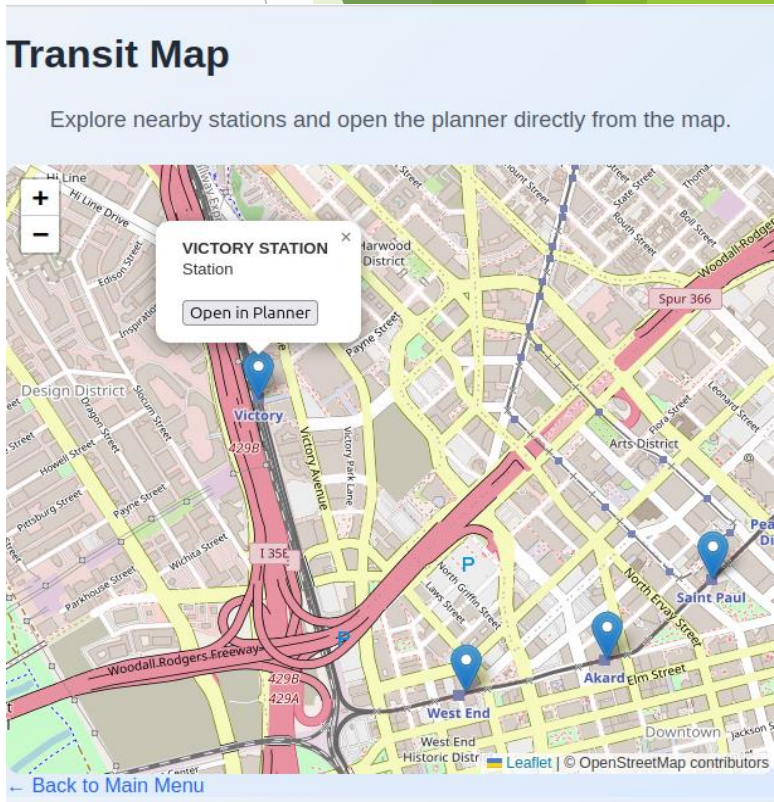


View Stations

- The user can view all stations in the region, pan, and zoom

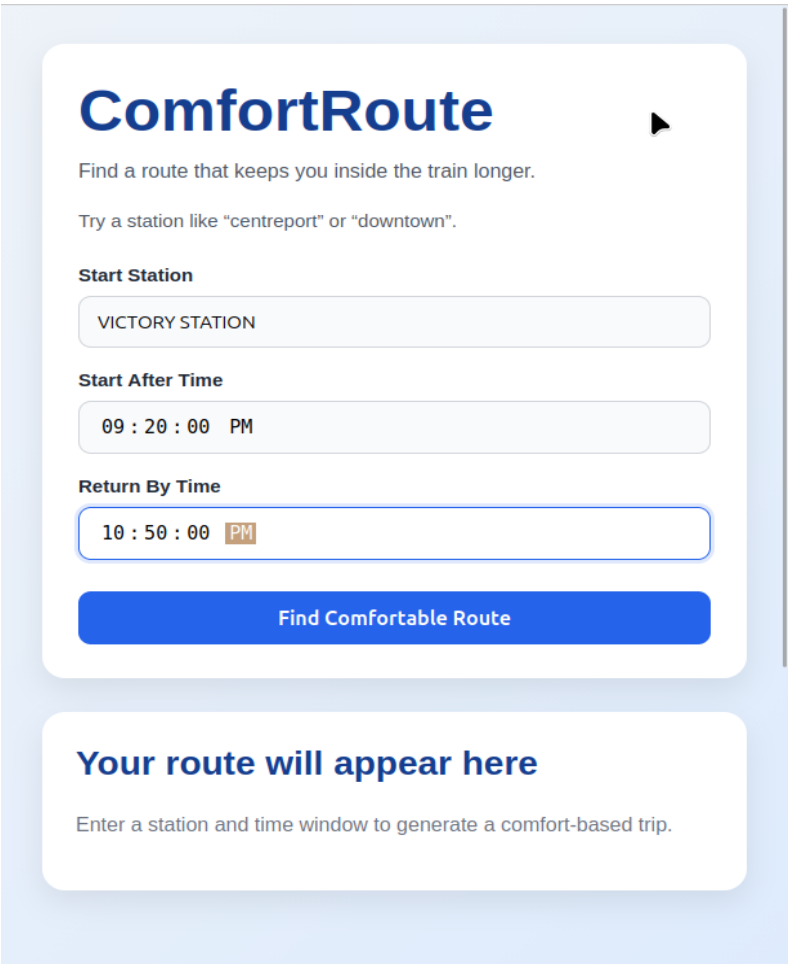
- The user can select a station to use in the route planner

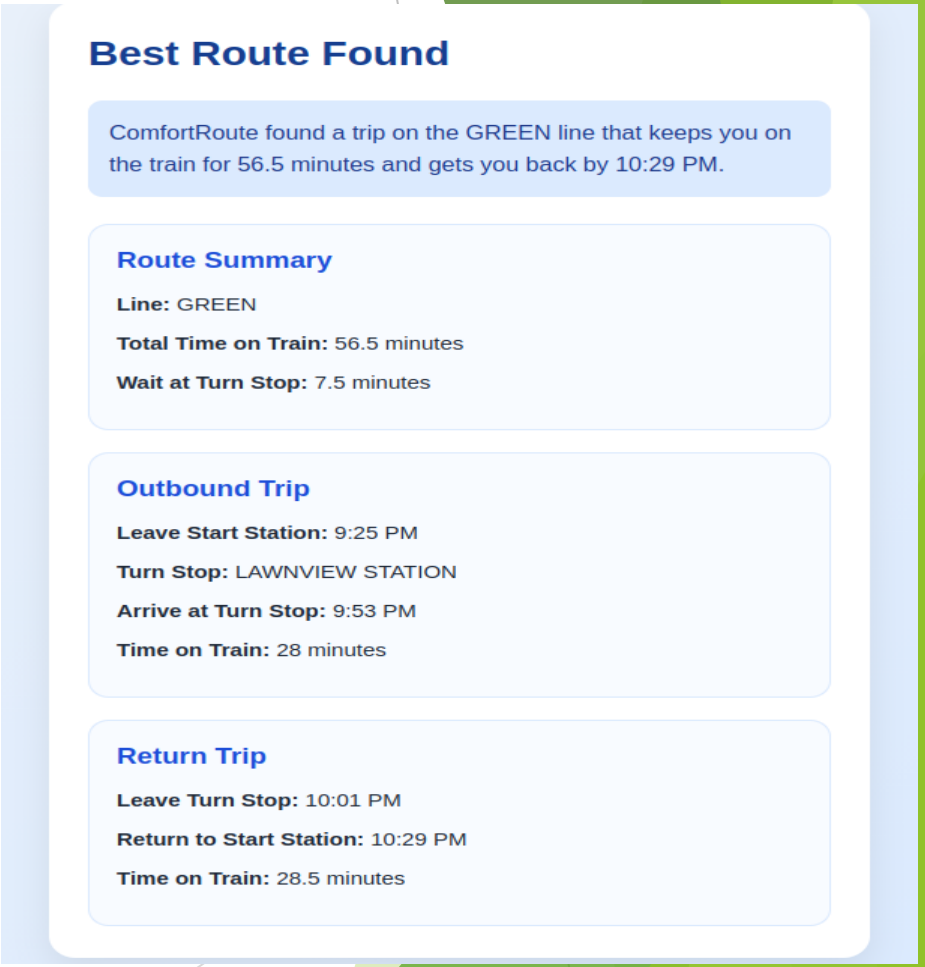




Finding the Best Route

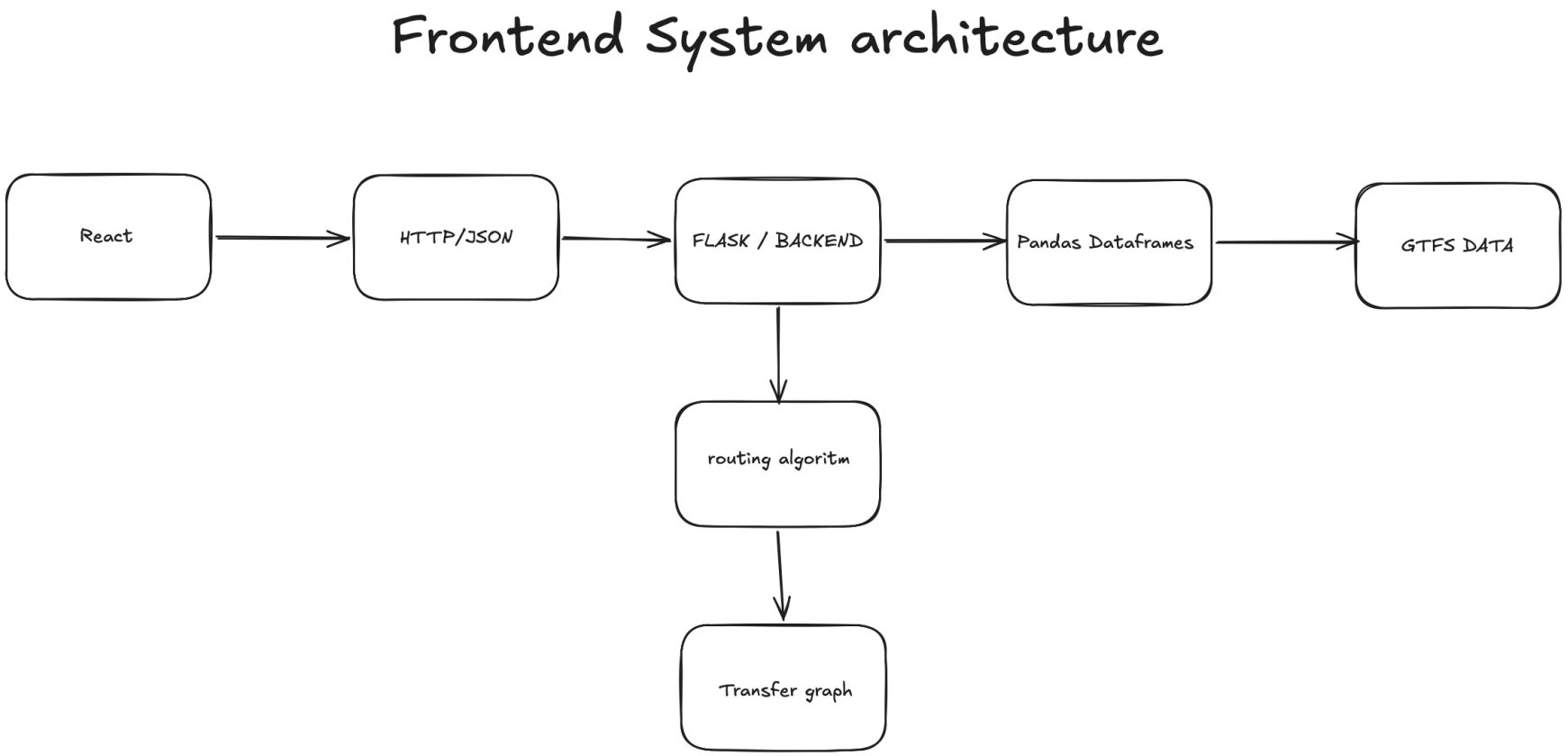
- Given a station, the app will recommend a route which will minimize time spent waiting at the platform.



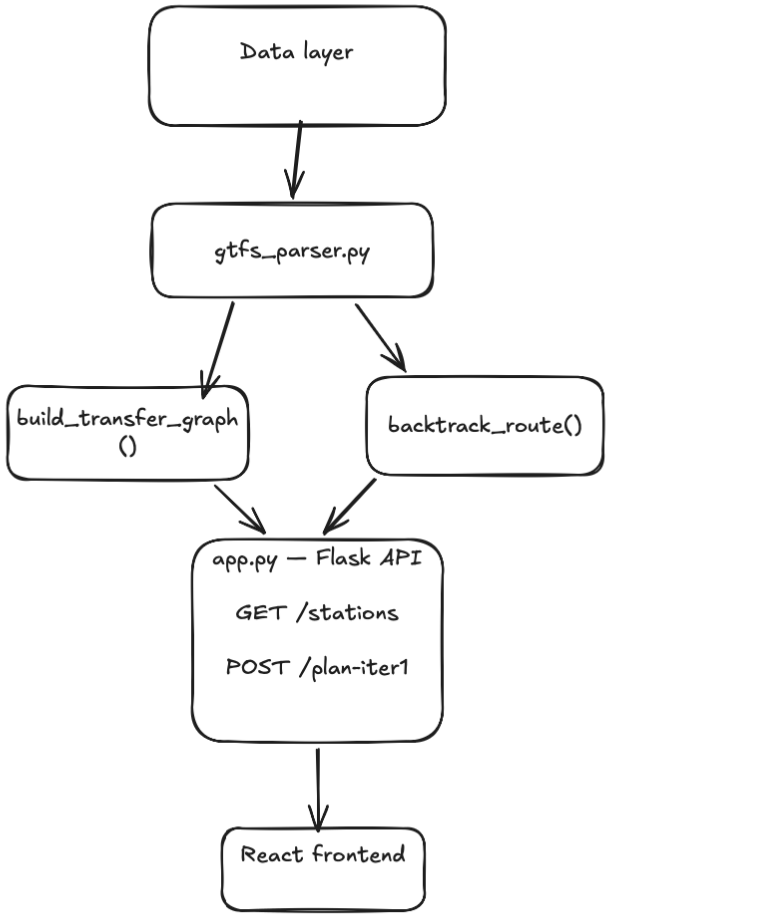


Live Demo

System Architecture: Frontend



System architecture: Backend



Technical Notes

- Data & Parsing
- Graph & Algorithm
- API Design

Risks

- Risk mitigation plan
  - Analyze users time constraints to prioritize speed of execution over accuracy for immediate requests
  - Compare OpenStreetMap data to publicly published data on DART website and update accordingly
  - Display service alerts to users, filter alerts for those relevant to user's trip and warn if route is affected.

Risk	Time Impact	Probability	Risk Exposure
(1) Algorithm does not run within required time	2 hours	50%	1 hour
(2) Station hours are not tagged/tagged improperly in OpenStreetMap	1 hour	25%	15 minutes
(3) GTFS schedules do not reflect reality on transit	4 hours	60%	2.5 hours

Next iteration

- Iteration 3:
  - Full routing algorithm
  - Implement settings for user customization
  - Improve map interaction (add colored route lines).
  - Show stations with indoor lobbies and their hours in the UI.

Thank you!

Any questions?